# *Homework 12*

*Xander Jhaveri*

## *Part A*

```
MAIN_PROGRAM    CODE
; Xander Jhaveri - xaj3 Part A
start:

; remaining code goes here
    clrw ; clears the w-register

    addlw 1 ; adds the value 1 to the w-reg
    addlw 1 ;
    addlw 1 ;
    addlw 1 ;
    addlw 1 ;
    addlw 1 ;
    addlw 1 ;
    addlw 1 ;
    addlw 1 ;
    addlw 1 ;

    ; Next step is to subtract 3 four times
    addlw -3 ; this instruction will subtract 3 from w-reg
    addlw -3
    addlw -3
    addlw -3
    ; repeat this addlw -3 line three more times

    ; Need to compare if the result is -2
    ; -2 in two's complement representation is 0xFE ('B 111111110)
    ; We do not have xnor as instruction so we need to use xor with 'B
00000001
    xorlw 1
    ; Xander Jhaveri - xaj3 Part A
fin:
    goto fin
        END                         ; directive 'end of program'
```

Listing 1: Source Code for Part A

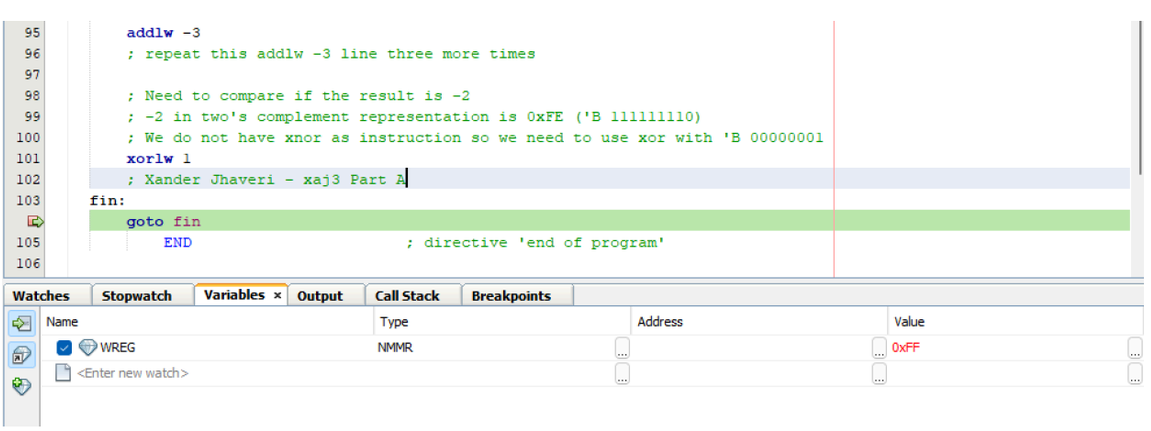


Figure 1: Screenshot of Variables Window for Part A

`WREG` equals `0xFF`, which means that it was equal to `-2` before the last instruction.

## *Part B*

```
MAIN_PROGRAM    CODE
; Xander Jhaveri - xaj3 Part B
start:

; remaining code goes here
    clrf my_result
    incf my_result,f ; make sure you increment my_result nine more times
    incf my_result,f ;
    incf my_result,f ;
    incf my_result,f ;
    incf my_result,f ;
    incf my_result,f ;
    incf my_result,f ;
    incf my_result,f ;
    incf my_result,f ;
    incf my_result,f ;

    movlw 3 ; this line writes 3 to w-reg
    subwf my_result,f ; this line will subtract 3 from my_result
    subwf my_result,f
    subwf my_result,f
    subwf my_result,f

    movlw 1
    xorwf my_result,1

    ; Xander Jhaveri - xaj3 Part B
fin:
    goto fin
        END                         ; directive 'end of program'
```

Listing 2: Source Code for Part B

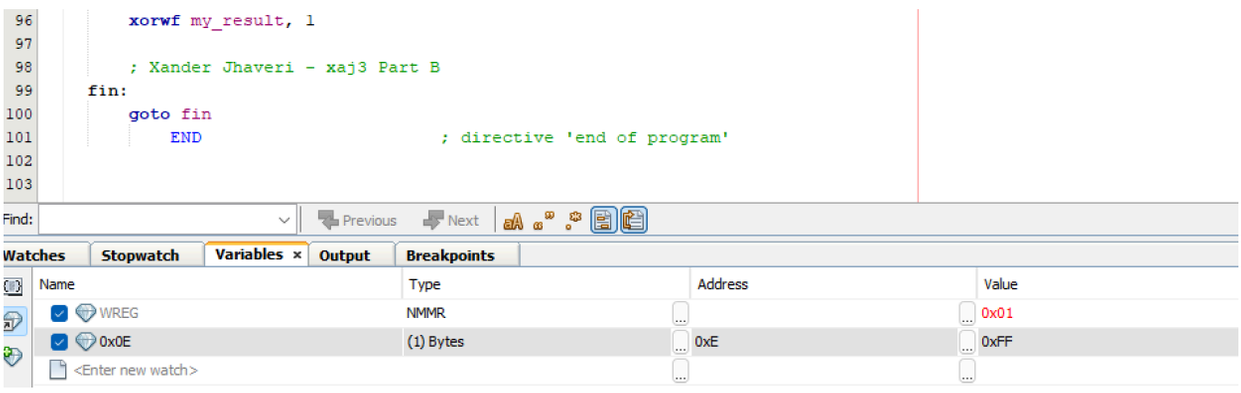


Figure 2: Screenshot of Variables Window for Part B

`0x0E` which is `my_result` is equal to `0xFF` which means that it was the complement of `WREG = 1` before the last instruction, so `my_result` was equal to `-2`.

## *Part C*

```
MAIN_PROGRAM    CODE
; Xander Jhaveri - xaj3 Part C
start:

; remaining code goes here
    clrf my_result
    movlw 0x0A
    movwf ctr ; makes the counter value equal to 10

loop1:
    incf my_result,f
    decfsz ctr,f
    goto loop1

    movlw 0x04
    movwf ctr
    movlw 0x03
loop2:
    subwf my_result, f
    decfsz ctr, f
    goto loop2

    movlw 0x01
    xorwf my_result, f

; Xander Jhaveri - xaj3 Part C
fin:
    goto fin

        END                         ; directive 'end of program'
```

Listing 3: Source Code for Part C

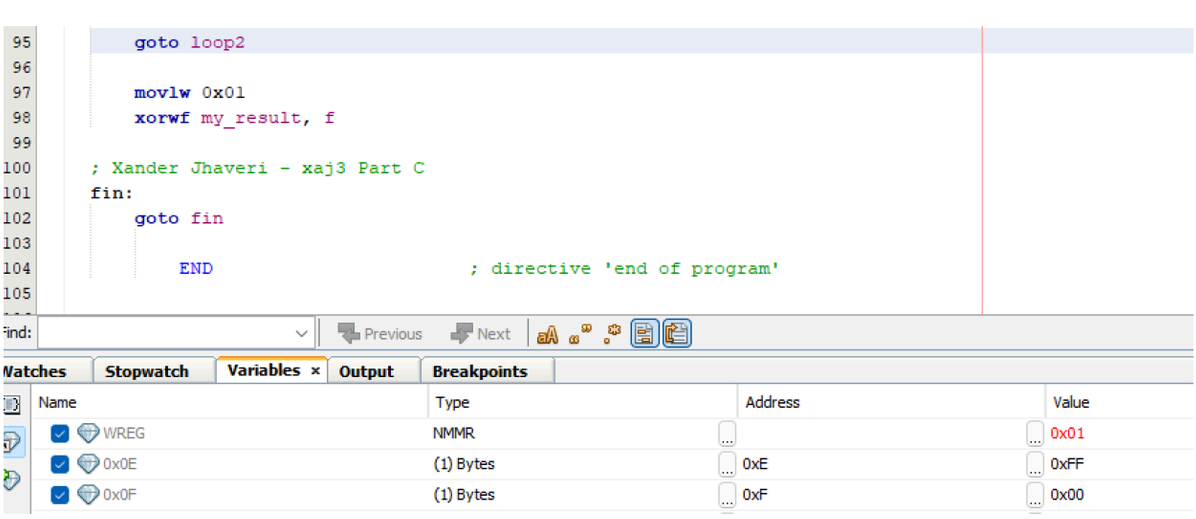


Figure 3: Screenshot of Variables Window for Part C

Again, `0x0E` or `my_result` is `0xFF` because it was equal to `-2`–the complement of `1`–before the last instruction.

## *Part D*

```
MAIN_PROGRAM    CODE
; Xander Jhaveri - xaj3 Part D
start:

; remaining code goes here
    clrf my_result
    movlw 0x0A
    movwf ctr ; makes the counter value equal to 10

    movf PORTB,w
    andlw 0x0F ; this will retain the least significant four bits (bits 3-0)
and
               ; it will remove the value in the bits 7-4

loop1:
    addwf my_result,f
    decfsz ctr,f
    goto loop1

    movlw 0x04
    movwf ctr

    swapf PORTB,w ; get the decrement amount from PORTB
    andlw 0x0F

loop2:
    subwf my_result,f
    decfsz ctr,f
    goto loop2

    movlw 0x01
    xorwf my_result, f

    ; Xander Jhaveri - xaj3 Part D
fin:
    goto fin

        END                         ; directive 'end of program'
```

Listing 4: Source Code for Part D

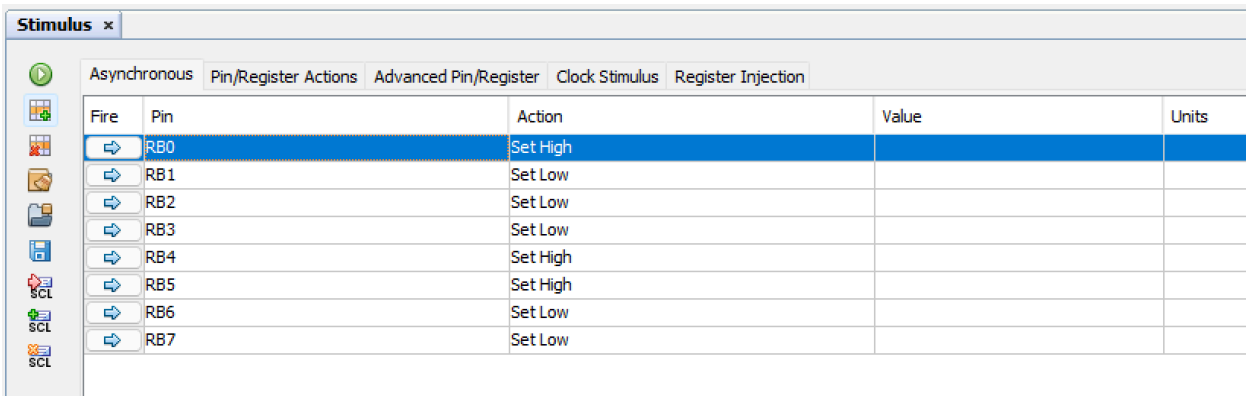


Figure 4: Screenshot of Stimulus for Part D

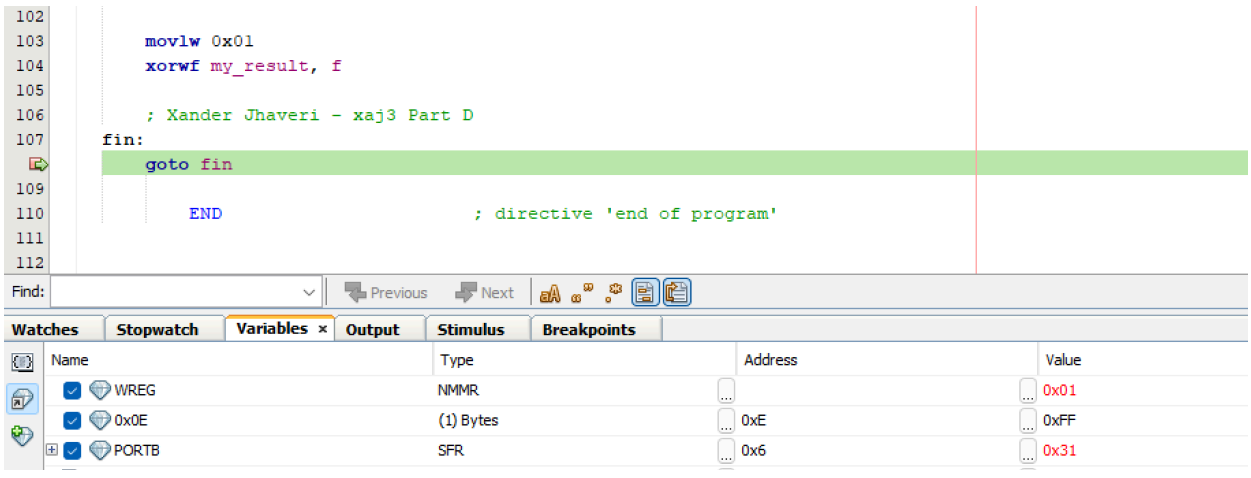


Figure 5: Screenshot of Variables Window for Part D

You can see that `PORTB` is equal to `0x31` which means the asynchronous stimulus properly set the most significant nibble of `PORTB` to `3` and the least significant to `1`.